

머신러닝 입문

강의 목차

- ❖ 머신러닝 기본 원리 및 주요 모델 이해
- ❖ Scikit-learn을 이용한 머신러닝 입문
- ❖ 머신러닝 실습
- ❖ 분류 모델
- ❖ 머신러닝 주요 개념
- ❖ Encoding
- ❖ Scaling

머신러닝 기본 원리 및 주요 모델 이해

머신러닝은 어떻게 스스로 학습하는가?

❖ 사례) 두 배우의 얼굴 분류

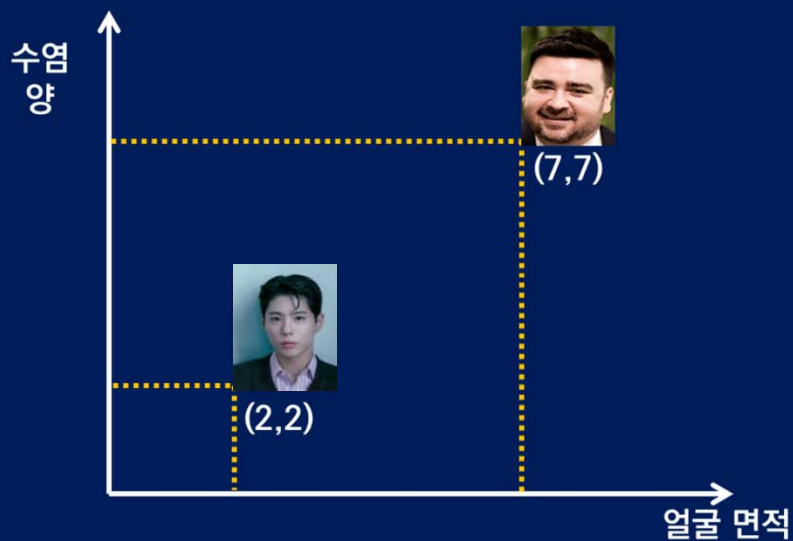


분류 기준: 얼굴 면적, 수염의 양

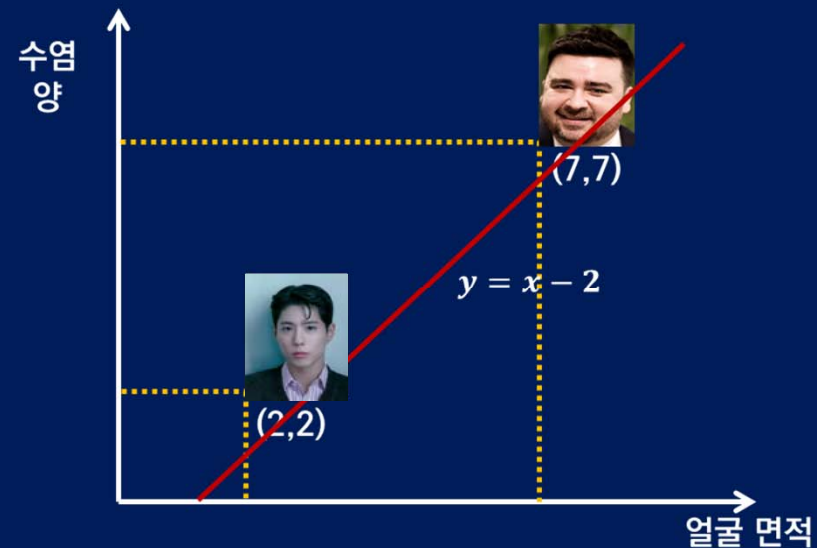
머신러닝은 어떻게 스스로 학습하는가?

❖ 사례) 두 배우의 얼굴 분류

1 데이터 준비



2 수학적 모델 생성



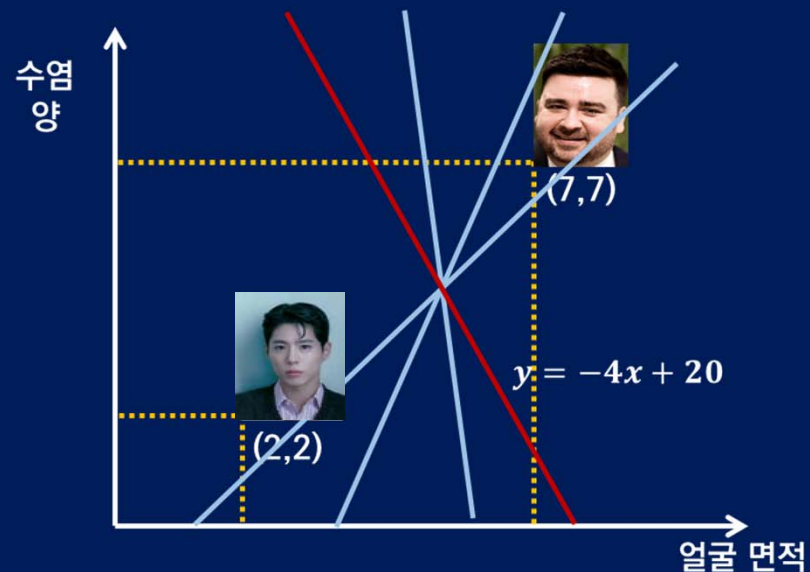
직선을 통해 두 사람의 위치가 분리되지 않음

분류 불가

머신러닝은 어떻게 스스로 학습하는가?

❖ 사례) 두 배우의 얼굴 분류

2 수학적 모델 생성



다량의 데이터

특징 파악

3 학습 완료



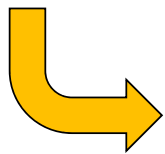
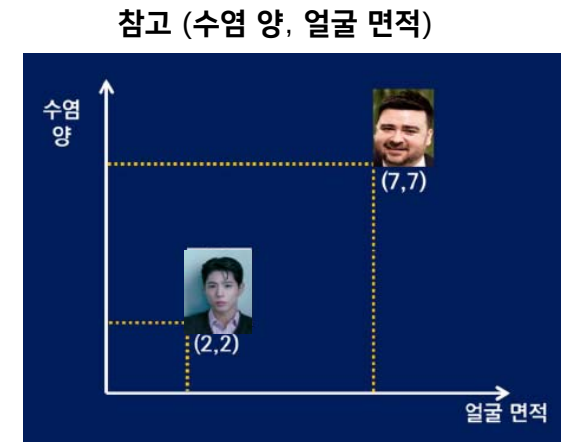
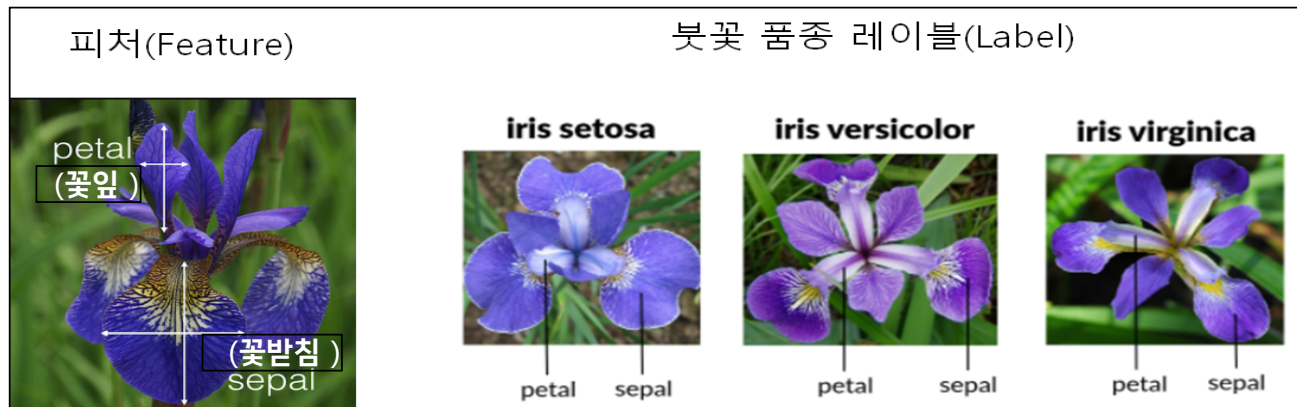
수학적 모델 구축

예측 수행

머신러닝 시작 예제

학습 예제 소개

❖ 학습예제의 데이터 셋



4개의 특성들(features) 클래스(class)

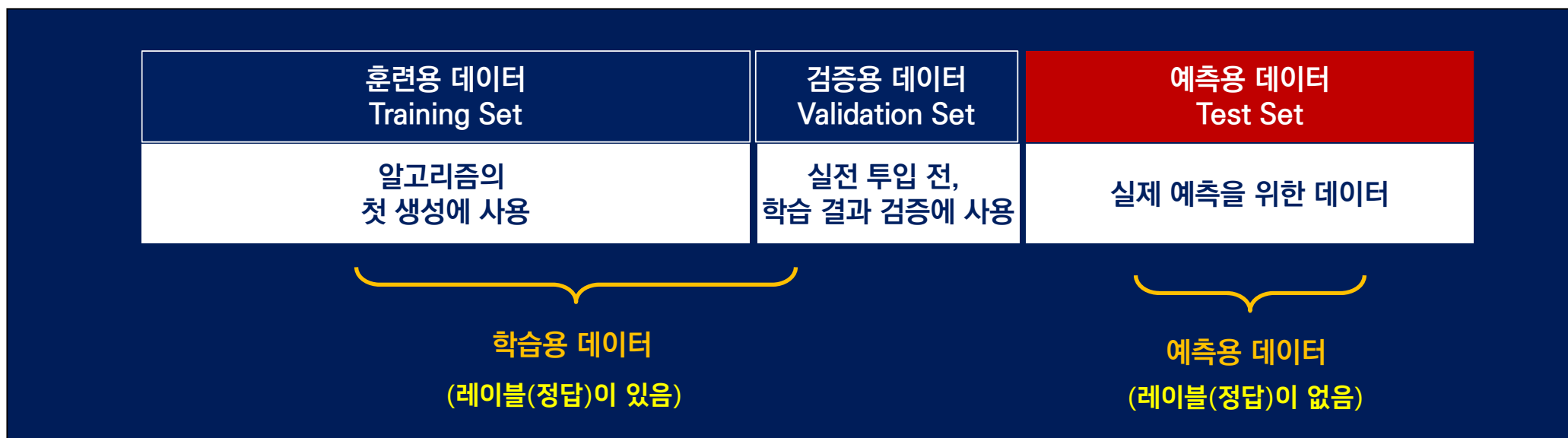
sepal_length	sepal_width	petal_length	petal_width	(Label, Target) species
5.8	4.0	1.2	0.2	setosa
5.1	2.5	3.0	1.1	versicolor
6.6	3.0	4.4	1.4	versicolor
5.4	3.9	1.3	0.4	setosa
7.9	3.8	6.4	2.0	virginica

0,1,2

머신러닝 시작 예제

머신러닝 데이터 셋

❖ 훈련용(train set), 검증용(validation set), 예측용 (test set)



sepal_length	sepal_width	petal_length	petal_width	species
5.8	4.0	1.2	0.2	setosa
5.1	2.5	3.0	1.1	versicolor
6.6	3.0	4.4	1.4	versicolor
5.4	3.9	1.3	0.4	setosa
7.9	3.8	6.4	2.0	virginica

sepal_length	sepal_width	petal_length	petal_width
5.8	4.0	1.2	0.2
5.1	2.5	3.0	1.1
6.6	3.0	4.4	1.4
5.4	3.9	1.3	0.4
7.9	3.8	6.4	2.0

머신러닝 학습의 종류

❖ 지도학습(Supervised Learning)

- 훈련데이터(train data set)에 타깃(target)또는 레이블(label)이라고 불리는 정답이 포함된 학습방식
- 훈련이 끝나면 정답이 포함되지 않은 새로운 데이터 셋(test data set)을 사용하여 정답을 예측하게 함

❖ 비지도학습(Unsupervised Learning)

- 훈련데이터(train data set)가 타깃(target)또는 레이블(label)이라고 불리는 정답이 포함하지 않음
- 훈련데이터에 별다른 가이드라인이 없어 기계학습모델이 스스로 학습하여 모델을 만듦

지도학습

❖ 개와 고양이의 분류 예시

레이블이 있는 데이터
(Labeled Data)



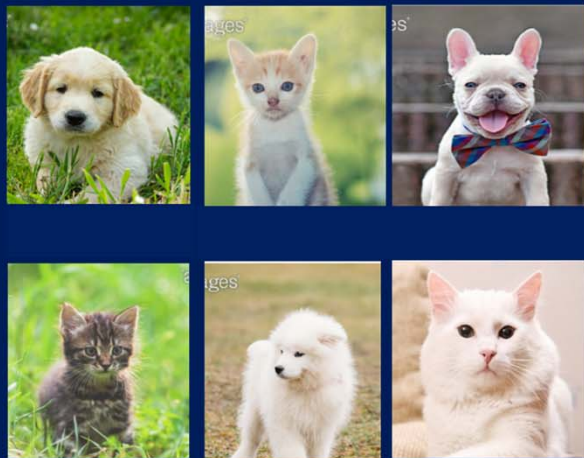
결과



비지도학습

❖ 개와 고양이의 군집화 예시

레이블이 없는 데이터
(Unlabeled Data)



결과

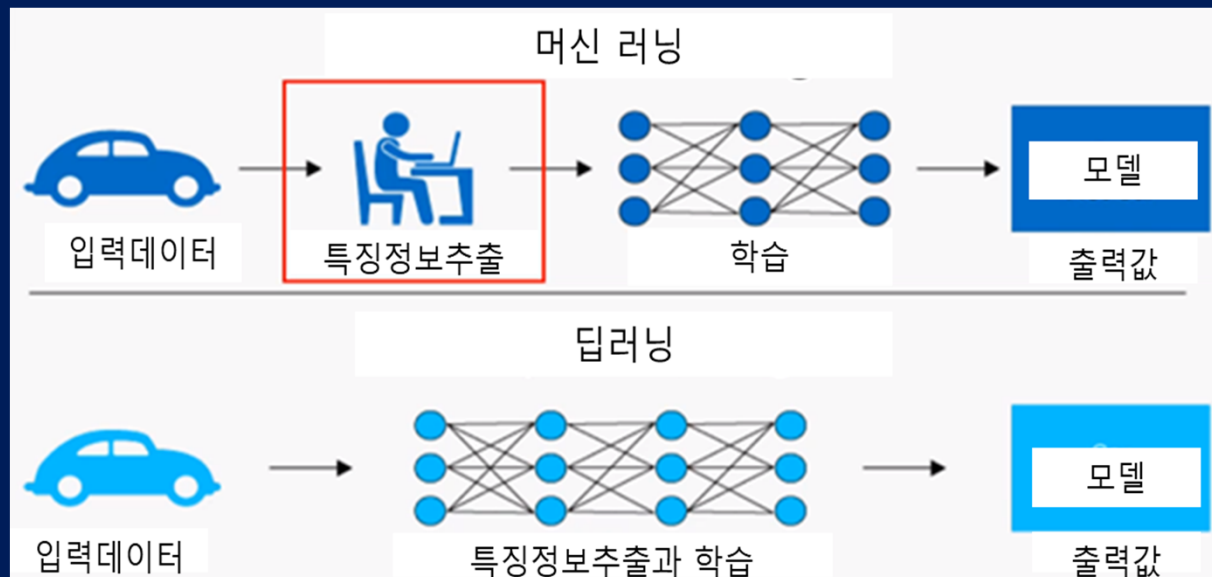


군집화

머신러닝과 딥러닝 비교

❖ 전통적 머신러닝 - 기계가 학습을 하기 전에 미리 데이터 특성을 파악

■ 머신러닝에서는 특징추출이 중요!



Scikit-learn을 이용한 머신러닝 입문

Scikit-Learn 라이브러리

- ❖ 파이썬 머신러닝 라이브러리 중 가장 많이 사용되는 라이브러리
- ❖ 텐서플로(TensorFlow), 케라스(Keras)같은 딥러닝 전문 라이브러리가 최근 각광받지만 파이썬 기반 대표 머신러닝 라이브러리는 사이킷런임
- ❖ 아나콘다 설치 시 사이킷런도 같이 설치되므로 import하여 사용

Scikit-Learn 라이브러리

❖ <https://scikit-learn.org/stable/>

[Install](#) [User Guide](#) [API](#) [Examples](#) [Community](#) [More](#)

scikit-learn

Machine Learning in Python

[Getting Started](#) [Release Highlights for 1.0](#) [GitHub](#)

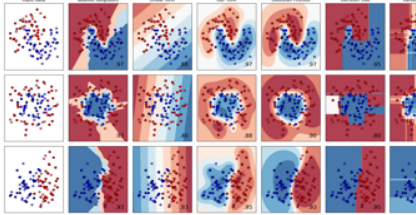
- Simple and efficient tools for predictive data analysis
- Accessible to everybody, and reusable in various contexts
- Built on NumPy, SciPy, and matplotlib
- Open source, commercially usable - BSD license

Classification

Identifying which category an object belongs to.

Applications: Spam detection, image recognition.

Algorithms: SVM, nearest neighbors, random forest, and more...



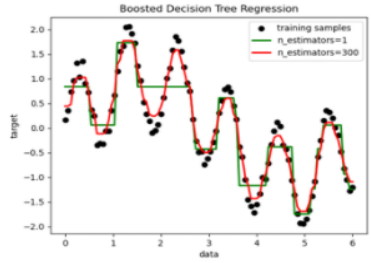
Examples

Regression

Predicting a continuous-valued attribute associated with an object.

Applications: Drug response, Stock prices.

Algorithms: SVR, nearest neighbors, random forest, and more...



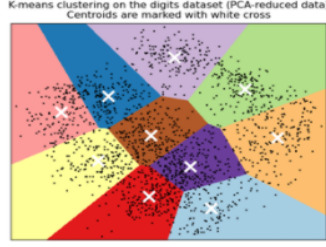
Examples

Clustering

Automatic grouping of similar objects into sets.

Applications: Customer segmentation, Grouping experiment outcomes

Algorithms: k-Means, spectral clustering, mean-shift, and more...



Examples

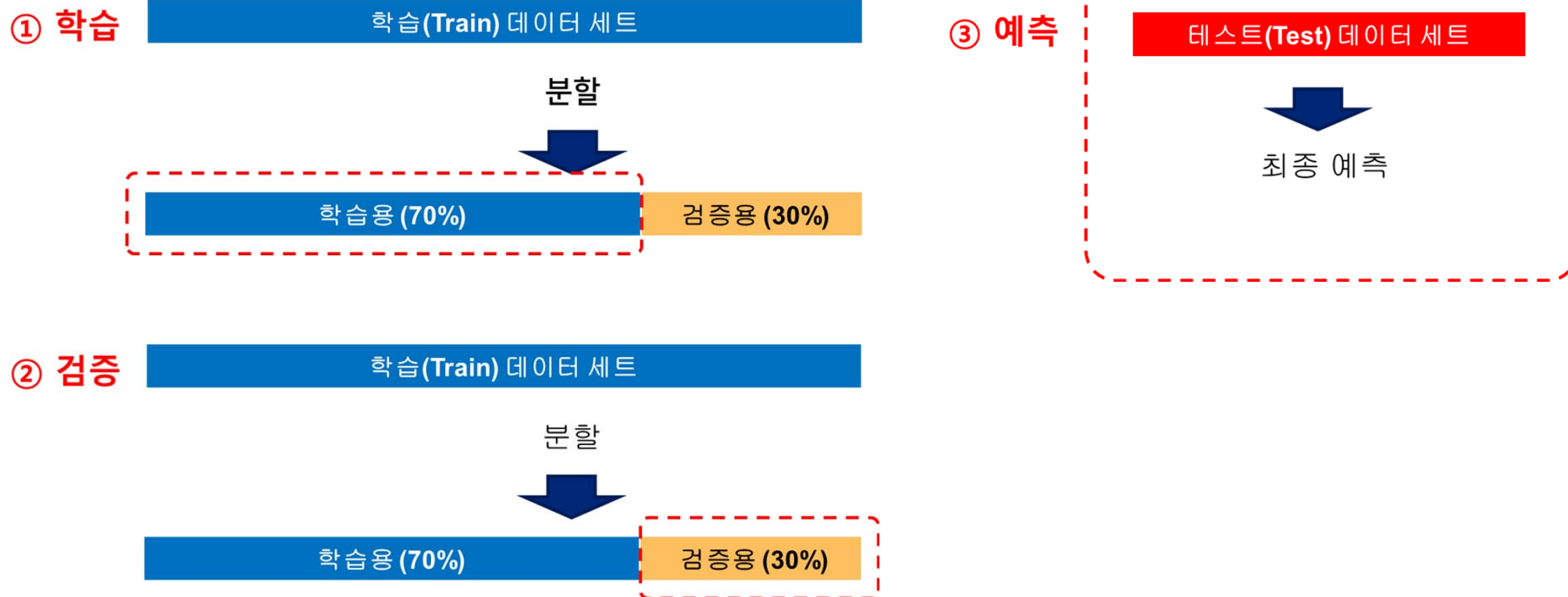
머신러닝의 전체 단계

1. 머신러닝 라이브러리 불러오기 (ex. 사이킷런)
2. 데이터 불러오기
3. 데이터 전처리 및 EDA를 통한 변수(feature) 설정
4. 학습데이터 셋(train data set)을 훈련 및 검증 데이터로 분리
5. 적합한 머신러닝 알고리즘의 선택하고 학습 (train)
6. 학습된 알고리즘에 검증 데이터에 적용하여 예측 (predict)
7. 예측값과 실제값을 비교해 오차를 측정하여 알고리즘의 성능 평가 (evaluation)
8. 3-8단계를 반복하며 알고리즘의 성능 고도화
9. 실제 예측을 원하는 데이터 셋(test data set)을 적용하여 최종 예측

머신러닝 시작 예제

머신러닝의 데이터 셋

❖ 훈련용(train set), 검증용(validation set), 예측용 (test set)



사이킷런 주요 모듈

분류	모듈명	설명
예제 데이터	sklearn.datasets	사이킷런에 내장되어 예제로 제공하는 데이터 세트
데이터 분리, 검증 & 파라미터 튜닝	sklearn.model_selection	교차 검증을 위한 학습용/테스트용 분리, 그리드 서치(Grid Search)로 최적 파라미터 추출 등의 API 제공
피처 처리	sklearn.preprocessing	데이터 전처리에 필요한 다양한 가공 기능 제공(문자열을 숫자형 코드 값으로 인코딩, 정규화, 스케일링 등)
	sklearn.feature_selection	알고리즘에 큰 영향을 미치는 피처를 우선순위 대로 선택션 작업을 수행하는 다양한 기능 제공
	sklearn.feature_extraction	텍스트 데이터나 이미지 데이터의 벡터화된 피처를 추출하는 데 사용됨.
		예를 들어 텍스트 데이터에서 Count Vectorizer 나 Tf-Idf Vectorizer 등을 생성하는 기능 제공. 텍스트 데이터의 피처 추출은 sklearn.feature_extraction.text 모듈에, 이미지 데이터의 피처 추출은 sklearn.feature_extraction.image 모듈에 지원 API가 있음.
피처 처리 & 차원 축소	sklearn.decomposition	차원 축소와 관련한 알고리즘을 지원하는 모듈임. PCA, NMF, Truncated SVD 등을 통해 차원 축소 기능을 수행할 수 있음

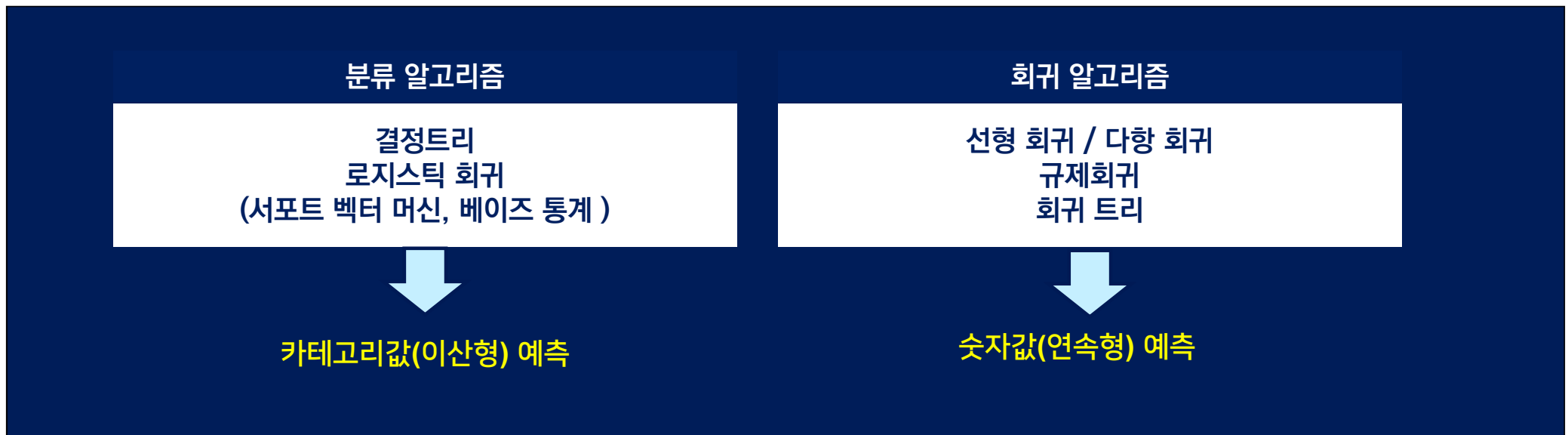
분류	모듈명	설명
평가	sklearn.metrics	분류, 회귀, 클러스터링, 페어와이즈(Pairwise)에 대한 다양한 성능 측정 방법 제공 Accuracy, Precision, Recall, ROC-AUC, RMSE 등 제공
ML 알고리즘	sklearn.ensemble	앙상블 알고리즘 제공 랜덤 포레스트, 에이다 부스트, 그라디언트 부스팅 등을 제공
	sklearn.linear_model	주로 선형 회귀, 릿지(Ridge), 라쏘(Lasso) 및 로지스틱 회귀 등 회귀 관련 알고리즘을 지원. 또한 SGD(Stochastic Gradient Descent) 관련 알고리즘도 제공
	sklearn.naive_bayes	나이브 베이즈 알고리즘 제공. 가우시안 NB, 다항 분포 NB 등.
	sklearn.neighbors	최근접 이웃 알고리즘 제공. K-NN 등
	sklearn.svm	서포트 벡터 머신 알고리즘 제공
	sklearn.tree	의사 결정 트리 알고리즘 제공
	sklearn.cluster	비지도 클러스터링 알고리즘 제공 (K-평균, 계층형, DBSCAN 등)
유틸리티	sklearn.pipeline	피처 처리 등의 변환과 ML 알고리즘 학습, 예측 등을 함께 묶어서 실행할 수 있는 유틸리티 제공

분류 모델

분류 모델

지도학습의 두 유형

- ❖ 분류 (classification) – 어떤 집단에 속하는 지 판별해 내는 것
- ❖ 회귀 (regression) – 주어진 값에 대한 결과값을 예측하는 것



분류 모델

분류 모델의 종류

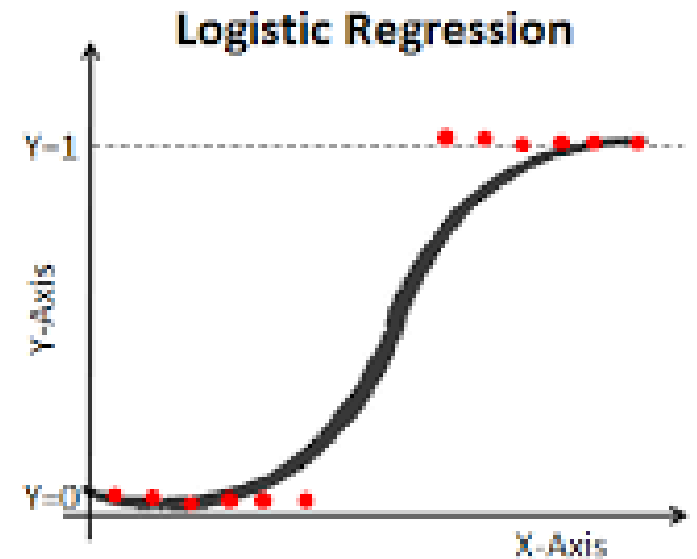
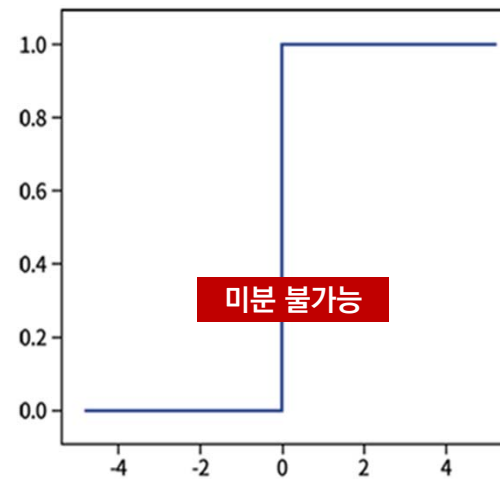
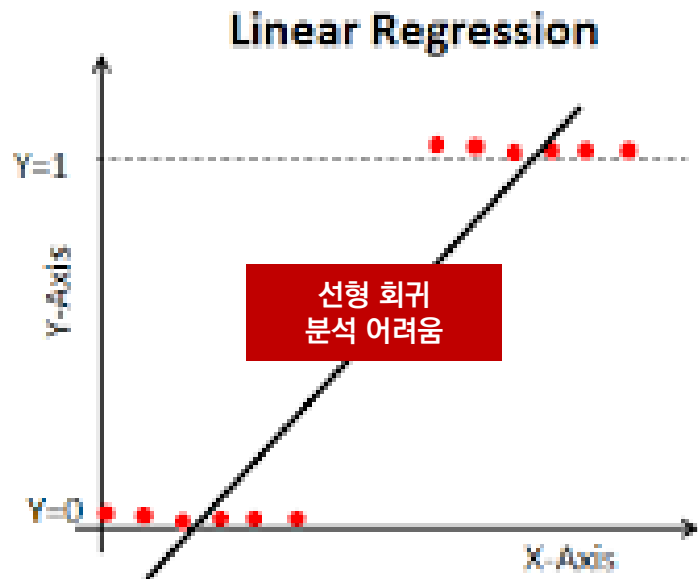
- ❖ 로지스틱 회귀 (Logistic Regression)
- ❖ K-nn
- ❖ 결정트리(Decision Tree, 의사결정나무)
- ❖ 앙상블 (서로 같거나/다른 알고리즘 간 결합)
 - Bagging - Random Forest, Extra Tree
 - Boosting - Gradient Boosting, xgBoost, LightGBM

분류 모델

로지스틱 회귀

❖ Target 데이터가 이산적 형태를 보일 때 사용하는 회귀 모델

- 연속형 독립변수(X), 범주형 종속변수(Y)
- 예측 값 $> 0 \rightarrow 1$ 로 분류, 예측 값 $< 0 \rightarrow 0$ 으로 분류

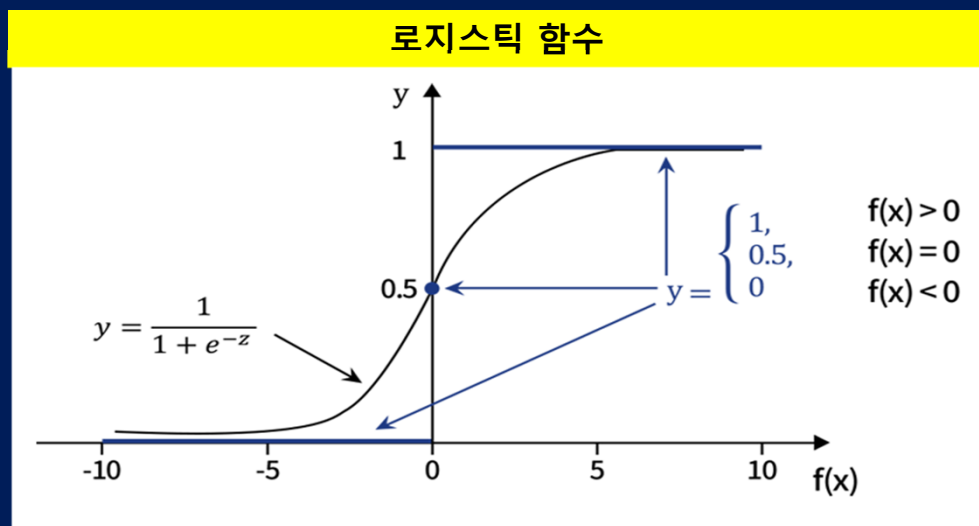


분류 모델

로지스틱 회귀

❖ 로지스틱 함수

■ 입력 값을 0이나 1에 근사한 출력 값으로 변환



$$y = \frac{1}{1 + e^{-f(x)}}$$

선형 회귀 분석 대입

$$= \frac{1}{1 + e^{-(w^T x + b)}}$$

$$\ln \frac{y}{1-y} = w^T x + b$$

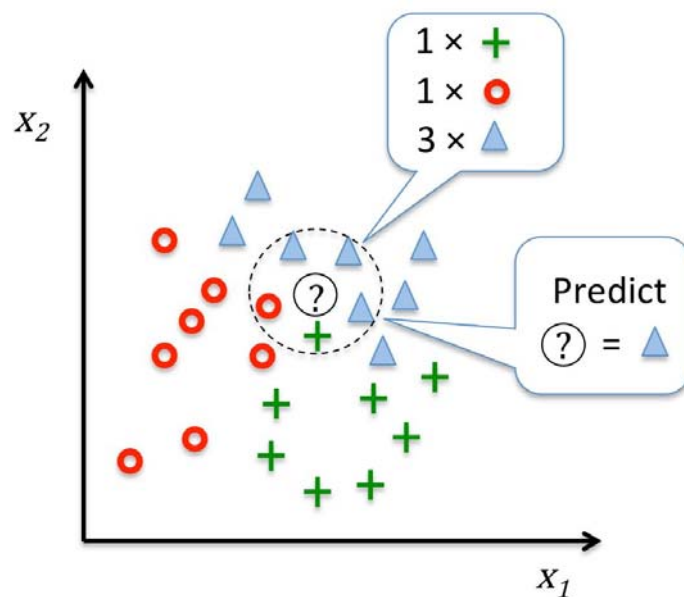
y	$f(x)$ 가 양의 값일 가능성
$1 - y$	$f(x)$ 가 음의 값일 가능성
$\frac{y}{1-y}$	$f(x)$ 가 양의 값일 상대적 가능성

오즈비
(Odds Ratio)

분류 모델

k-NN

- ❖ k 에 해당하는 숫자, 거리 메트릭 선택
- ❖ 분류하고자 하는 샘플에 대한 k개의 근접한 이웃을 찾음
- ❖ 다수결 투표 방식으로 분류 레이블을 할당함



분류 모델

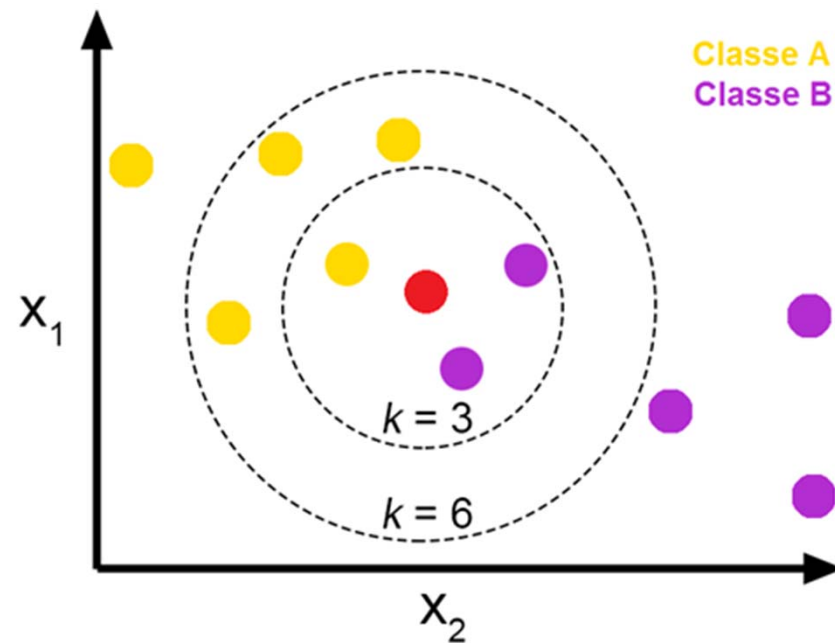
k-NN

- ❖ k-Nearest Neighbor
- ❖ k를 알맞게 선택하는 것이 아주 중요
- ❖ 데이터 features 에 대한 적당한 거리 metric 선택 필요
- ❖ 많이 쓰는 거리 함수
 - Minkowski distance
 - Euclidean distance
 - Manhattan distance

분류 모델

k-NN

- ❖ 데이터 분류 시 주변의 데이터를 살펴본 뒤
더 많은 데이터가 포함된 범주로 분류하는 방식



분류 모델

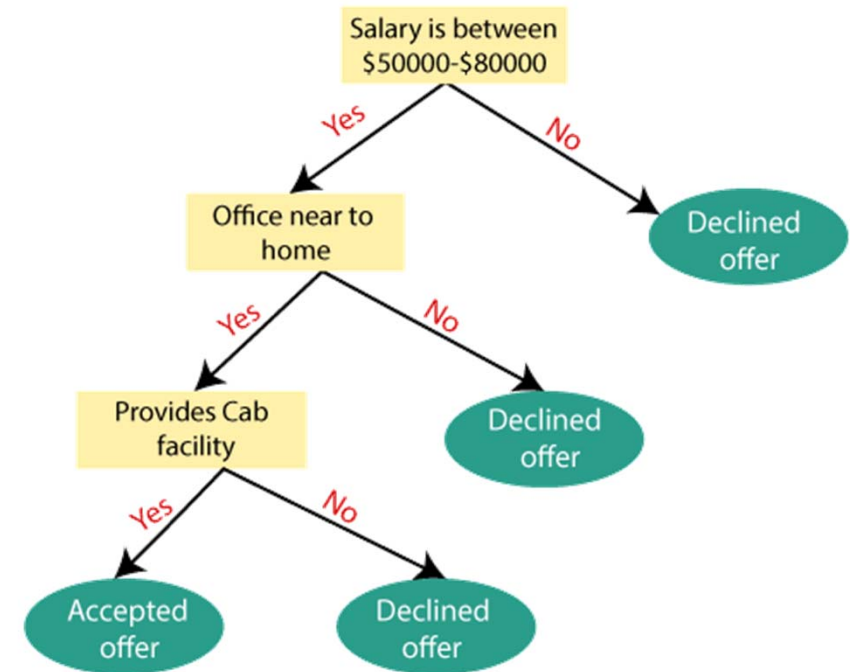
k-NN

- ❖ 알고리즘이 간단하여 구현하기 쉬움
- ❖ 수치 기반 데이터 분류 작업에서 성능이 좋음
- ❖ 학습 데이터의 양이 많으면 분류 속도가 느려짐
- ❖ 차원(벡터)의 크기가 크면 계산량이 많아짐

분류 모델

결정트리

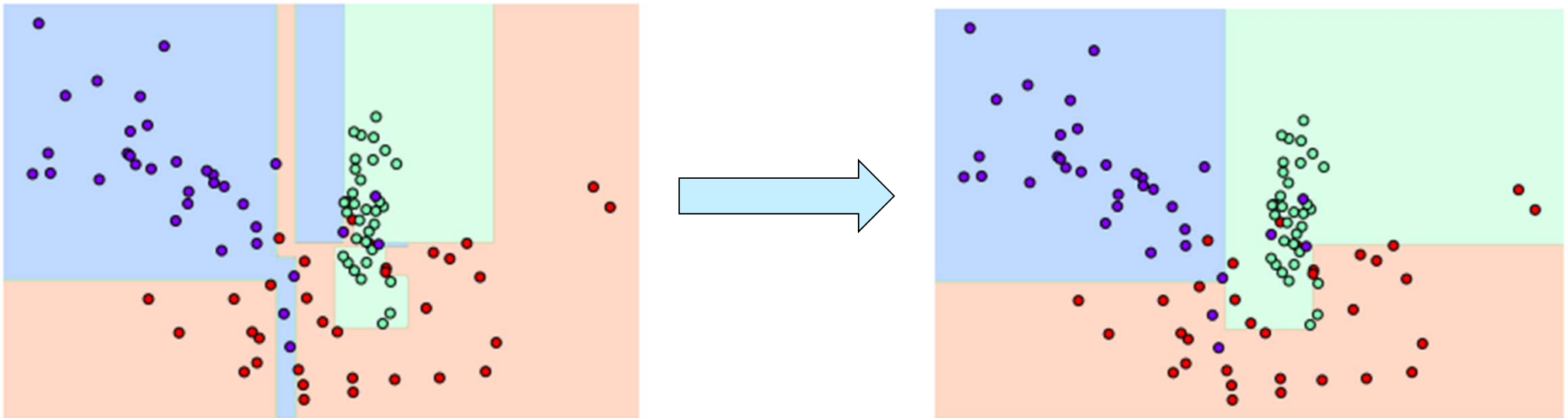
- ❖ 결정트리는 매우 쉽고, 스케일링이나 정규화 등 사전 데이터 가공의 영향이 적음
- ❖ 예측 성능을 향상시키기 위해 **복잡한 규칙 구조**를 가져야 하고 이로 인한 **과적합**이 발생, 예측성능 떨어질수도 있음
 - **depth**가 깊어질수록 결정트리의 예측성능은 저하될 수 있음



결정트리

❖ 복잡하고 엄격한 학습 규칙으로 인한 과적합 사례

- 결정트리의 max depth를 제한하거나, 말단노드의 데이터 수를 완화하는 방식으로 해결



분류 모델

결정트리

❖ 불순도 계산 방법

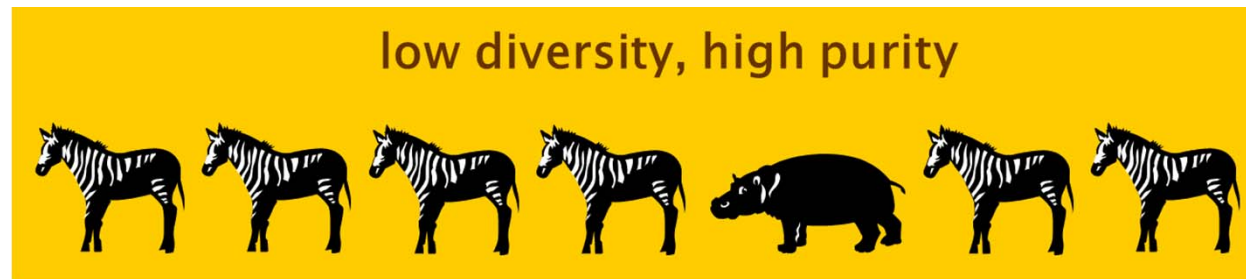
- 지니 지수
- 엔트로피
- 분류오류

❖ 예) 지니지수

$$I(A) = 1 - \sum_{k=1}^M p_k^2$$



$$G = 1 - (3/8)^2 - (3/8)^2 - (1/8)^2 - (1/8)^2 = .69$$



$$G = 1 - (6/7)^2 - (1/7)^2 = .24$$

앙상블 모델

회귀 앙상블 모델

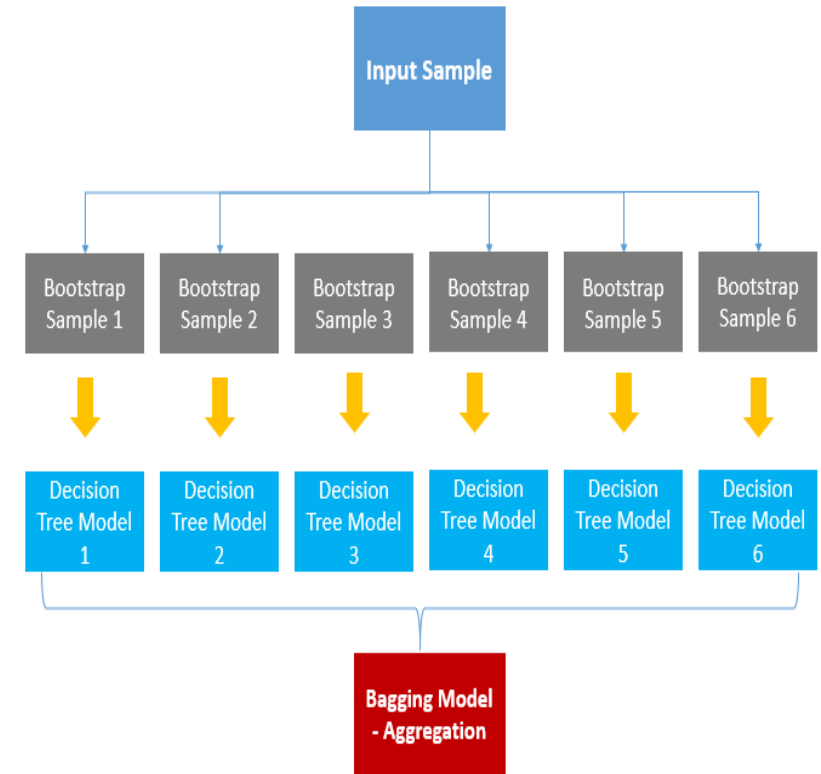
앙상블 모델

- 여러 개의 모델을 결합하여 사용하는 모델
 - Regression, Classification 모두 지원
 - 회귀(단일 모델들의 결과를 평균하여 예측), 분류(단일 모델들의 결과에 투표하여 예측)
- 머신러닝의 앙상블은 크게 배깅(Bagging)과 부스팅(Boosting) 두 가지로 구분
 - 둘다 상대적으로 약한 Decision Tree를 조합
 - 단, 배깅(Bagging)은 병렬적으로 여러 개의 독립적 결정트리를 만들어 결합한다면(평균)
 - 부스팅(Boosting)은 직렬적으로 1개의 모델을 계속 업데이트 시켜가는 방식

앙상블 모델 - 배깅

배깅 (Bagging)

- Bagging: Bootstrap Aggregation
 - Bootstrap으로 각 결정트리를 학습시킨 후 그 결과물을 집계(Aggregation)하는 방법



앙상블 모델 - 배깅

배깅 (Bagging)

- 부트스트랩(Bootstrap)

- 샘플을 여러 번 뽑는 방법
- n 개의 결정트리가 서로 다른 데이터로 학습토록 하기 위하여
- 원본 데이터셋에서 중복을 허용하여 무작위로 m 개의 데이터를 선택하는 기법



앙상블 모델 - 배깅

배깅 (Bagging)

- 랜덤 포레스트 (Random Forest)

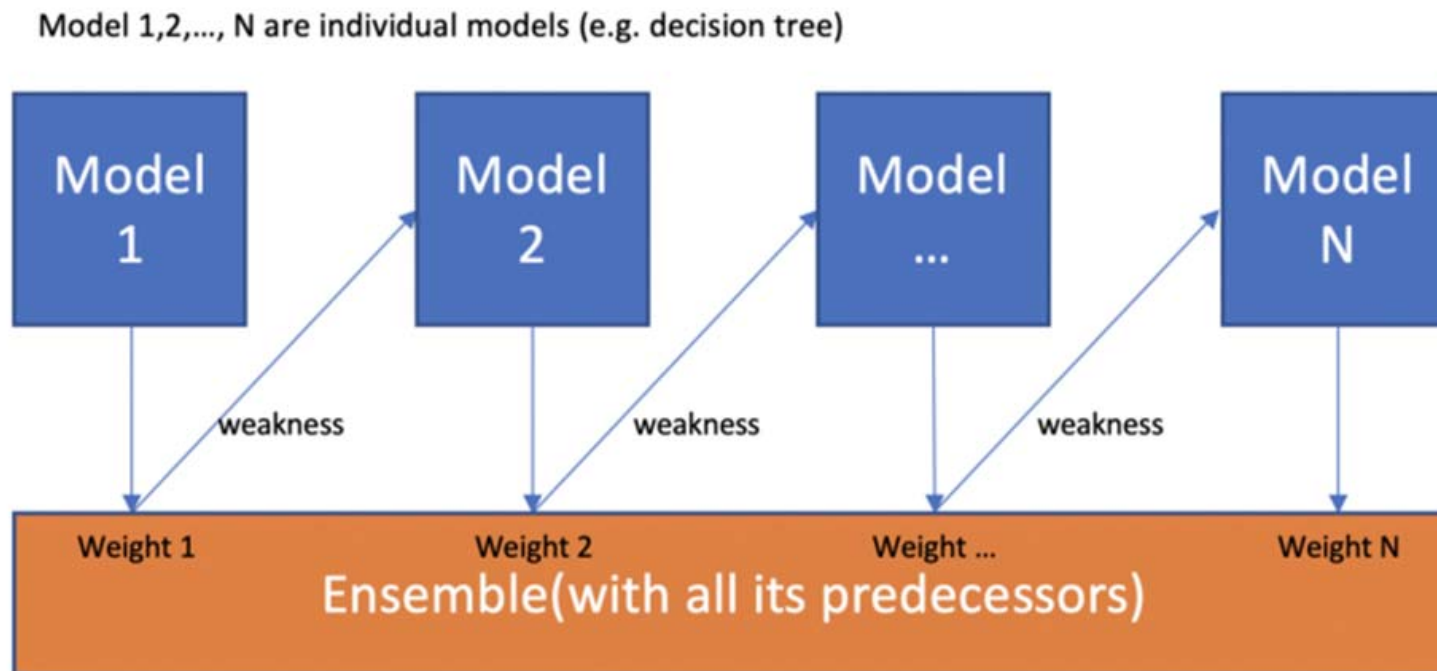
- 결정트리(decision tree)를 n개 결합한 모델
- 데이터 분할 최적화를 위해 부트스트랩 기법을 이용

- 엑스트라 트리 (ExtraTrees)

- 랜덤 포레스트의 변형 형태
- Extremely Randomized 학습법 (샘플수, feature 설정이 모두 random)
- 부트스트랩은 사용하지 않음: (bootstrap=False)로 파라미터 초기값 설정
- 그 외는 랜덤 포레스트와 동일한 앙상블 모델
- 랜덤 포레스트는 데이터를 최적 분할하지만(bootstrap) 엑스트라 트리는 임의 분할하여 랜덤 포레스트보다 속도가 빠름

부스팅 (Boosting)

- 상대적으로 약한 Decision Tree를 조합해서 사용
- 약한 예측 모형들의 학습 에러에 가중치를 두고, 순차적으로 다음 학습 모델에 반영하여 점점 강한 예측모형을 만드는 것



부스팅 (Boosting)

- **XGBoost (eXtreme Gradient Boosting)**

- Boosting 기법을 이용하여 구현한 대표적 알고리즘
- 성능과 자원 효율이 좋아서 많이 사용되는 알고리즘
- [XGBoost Documentation — xgboost 2.0.0 documentation](#)

- **LightGBM (Light Gradient Boosting Model)**

- Boosting 기법을 이용하여 구현한 최신 알고리즘
- 학습시간이 짧아 XGBoost 보다 속도가 빠름(light)
- 파라미터의 수가 매우 많음
- [Welcome to LightGBM's documentation! — LightGBM 4.1.0.99 documentation](#)

머신러닝 주요 개념

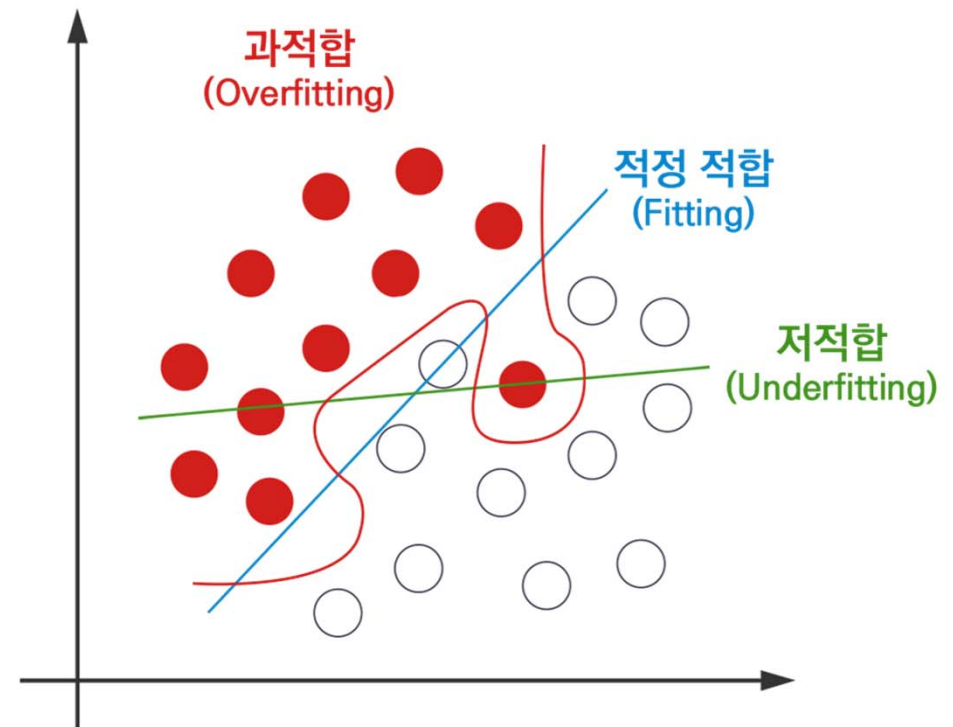
과적합(Overfitting)과 저적합(Underfitting)

❖ 과적합 (과대적합)

- 학습 모델이 현재의 훈련데이터에 지나치게 적합되어 학습하여 복잡한 모델을 생성
- 일반화 (새로운 데이터에 대응) 어려움

❖ 저적합 (과소적합)

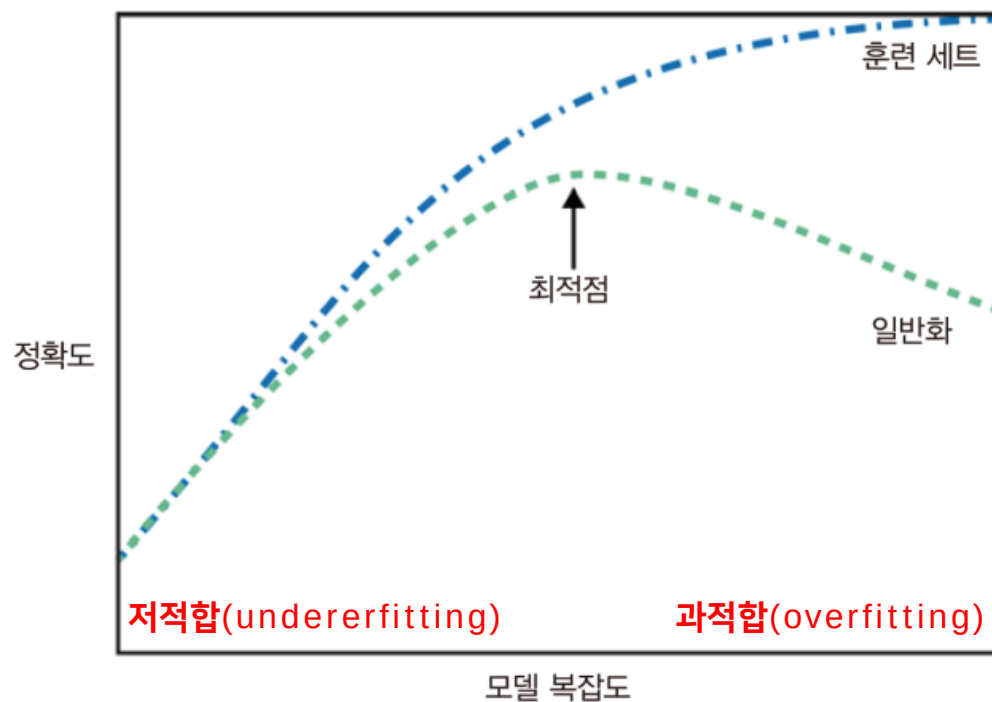
- 주어진 훈련데이터도 충분히 학습을 하지 못하여 (또는 훈련데이터가 충분치 않아서) 너무 간단한 모델을 생성
- 예측 성능 떨어짐



머신러닝 주요 개념

과적합(Overfitting)과 저적합(Underfitting)

❖ 모델 복잡도와 예측 정확도의 관계



분류모델 평가 지표

❖ 오차 행렬 (Confusion Matrix)

- 이진분류에서 성능지표로 활용

	예측: Negative	예측: Positive
실제: Negative	TN (True Negative)	FP (False Positive)
실제: Positive	FN (False Negative)	TP (True Positive)

분류모델 평가 지표

❖ 정확도 (Accuracy)

- 정확히 예측한 수를 전체 샘플 수로 나눈 값

- $\text{정확도} = \frac{\text{예측 결과가 정확한 데이터 건수}}{\text{전체 예측 데이터 건수}}$

$$= \frac{TN + TP}{TN + FP + FN + TP}$$

	예측: Negative	예측: Positive
실제: Negative	TN (True Negative)	FP (False Positive)
실제: Positive	FN (False Negative)	TP (True Positive)

분류모델 평가 지표

	예측: Negative	예측: Positive
실제: Negative	TN (True Negative)	FP (False Positive)
실제: Positive	FN (False Negative)	TP (True Positive)

❖ 정밀도(precision)

- 양성 예측(FP+TP) 중 진짜 양성(TP)의 수
- 예측을 positive 로 한 대상 중 정확하게 한 건수의 비율
 - 예) (코로나) 양성으로 나온 사람 중 실제 양성자의 비율
- positive 예측성능을 더욱 정밀하게 봄

$$\text{정밀도} = \frac{TP}{FP+TP}$$

❖ 재현율(recall)

- 실제 양성 (FN+TP) 중, 양성 예측 (TP)의 수
- 실제 positive 대상 중 정확하게 positive로 예측 한 건수의 비율
 - 예) (코로나) 실제 양성인 사람 중 양성으로 정확하게 예측된 비율
- 민감도(sensitivity), 적중률(hit rate)

$$\text{재현율} = \frac{TP}{FN+TP}$$

분류모델 평가 지표

❖ F1-Score

$$\blacksquare 2 \cdot \frac{precision \cdot recall}{precision + recall} = \frac{2}{\frac{1}{precision} + \frac{1}{recall}} \quad F = 2 \cdot \frac{\text{정밀도} \cdot \text{재현율}}{\text{정밀도} + \text{재현율}}$$

■ 정밀도(precision)와 재현율(recall)의 조화평균

- 현실적으로 정밀도와 재현율을 함께 높이기 어렵음
- 따라서 조화평균을 통해 두 지표의 종합적 성능을 표현
- precision과 recall이 모두 높으면 높아짐
- 두 지표의 합이 동일하여도, 한 값이 너무 적으면 F1_score는 낮아짐
 - ✓ precision = 0.9, recall = 0.1 → f1_score = 0.18
 - ✓ precision = 0.6, recall = 0.4 → f1_score = 0.24

■ 0.0 ~ 1.0 사이의 값을 가짐

Encoding

예측력 높이기 도전!

데이터 인코딩

❖ 레이블 인코딩

■ 문자열 값을 숫자형 카테고리 값으로 변환

- Sex: [남자, 여자] → [0,1]
- Embarked: [C, S, Q] → [0,1,2]

레이블 인코딩의 문제점?

- 레이블 인코딩 후 숫자값이 부여되는데 이를 머신러닝에 바로 사용하면, 예측 성능이 떨어질 수 있음
 - 냉장고 1, 선풍기2 → 1보다 2가 크다는 의미가 부여되기 때문 (실제로는 단순한 구분용일 뿐인데..)
- 이러한 이유로 레이블 인코딩은 회귀분석과 같은 머신러닝에서 사용하지 않아야 함
- 레이블 인코딩의 이러한 문제점을 해결하기 위한 방법이 바로 "원-핫 인코딩" 임!

예측력 높이기 도전!

데이터 인코딩

❖ 원-핫 인코딩

- 새로운 feature 를 추가하고 고유 값에 해당하는 칼럼에만 1을 표시, 나머지는 모두 0을 표시하는 방식
- Pandas의 **get_dummies()** 또는 사이킷런의 OneHotEncoder() 로 수행

레이블 인코딩		원-핫 인코딩					
상품 분류	상품분류	상품 분류_ TV	상품 분류_ 냉장고	상품 분류_ 전자렌지	상품 분류_ 선풍기	상품 분류_ 컴퓨터	상품 분류_ 믹서
TV	0	1	0	0	0	0	0
냉장고	1	0	1	0	0	0	0
전자렌지	2	0	0	1	0	0	0
컴퓨터	4	0	0	0	0	1	0
선풍기	3	0	0	0	1	0	0
선풍기	3	0	0	0	1	0	0
믹서	3	0	0	0	0	0	1
믹서	5	0	0	0	0	0	1

Scaling

Scaling (스케일링)

❖ 스케일링의 목적

- feature들의 값을 있는 그대로 넣었을 때, 각 값의 평균과 분산, 최대 최소 값이 제각각
- feature간 비교/분석이 어렵고 따라서 학습 모델을 잘 만들지 못할 수 있음
- 표준화 또는 정규화를 통해 각 feature값의 분포나 범위를 제한해 줌으로써 모델의 학습 성능을 높일 수 있음

Scaling (스케일링)

❖ 표준화

- 데이터 피쳐 각각이 평균이 0, 분산이 1인 가우시안 정규분포를 가진 값으로 변화하는 것
- 선형회귀, 로지스틱회귀, 서포트벡터머신(SVM)은 데이터가 정규분포를 가진다고 가정하기 때문에 사전에 표준화를 제공하는 것은 예측 성능 향상에 중요한 요소가 될 수 있음.
- 왜도가 심한 데이터들의 경우 표준화 진행을 통해 예측력 향상을 기대함.

$$x_{new} = \frac{x - \mu}{\sigma}$$

표준화 공식

Scaling (스케일링)

- ❖ 정규화 (Normalization)

 - 데이터를 0과 1사이의 값으로 변환함

- ❖ 서로 다른 피처의 크기를 동일한 크기로 변환해 주는 개념

 - 예) 가격 0원~ 50,000,000원

 - 예) 몸무게 0~100kg

- ❖ 모두 동일한 크기 단위로 비교 하기 위해 최소 0~최대1값으로 변화하는 것

$$x_{new} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Scaling (스케일링)

❖ 데이터 전처리 방법들

- StandardScaler(), MinMaxScaler(), RobustScaler(), Normalize() 등

❖ fit() 과 transform()

- fit(): 스케일링을 위한 기준정보 설정 (ex. 데이터 세트의 최대값/최소값 설정)
- transform() : 설정된 정보를 바탕으로 데이터 변환(scaling)

❖ 데이터별 적용

- 훈련 데이터: fit() & transform()
- 테스트 데이터: 훈련 데이터 기준에 맞추어서 transform()
- y 데이터: 전처리 하지 않음.

Scaling (스케일링)

❖ 코드 예시

- `scale = StandardScaler().fit(X_train)`
- `X_train_scaled = scale.transform(X_train)`
- `X_test_scaled = scale.transform(X_test)`

Wrap-Up